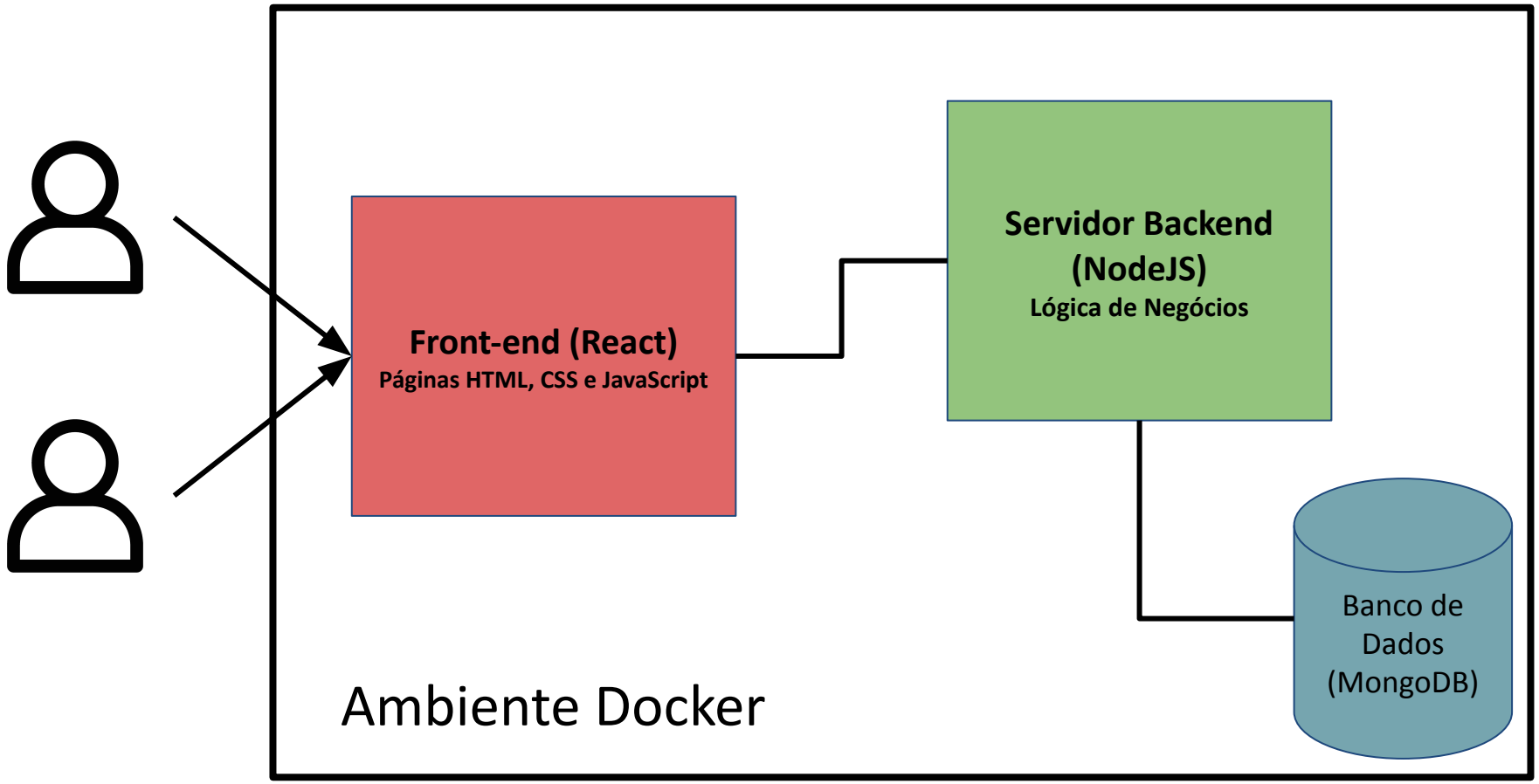
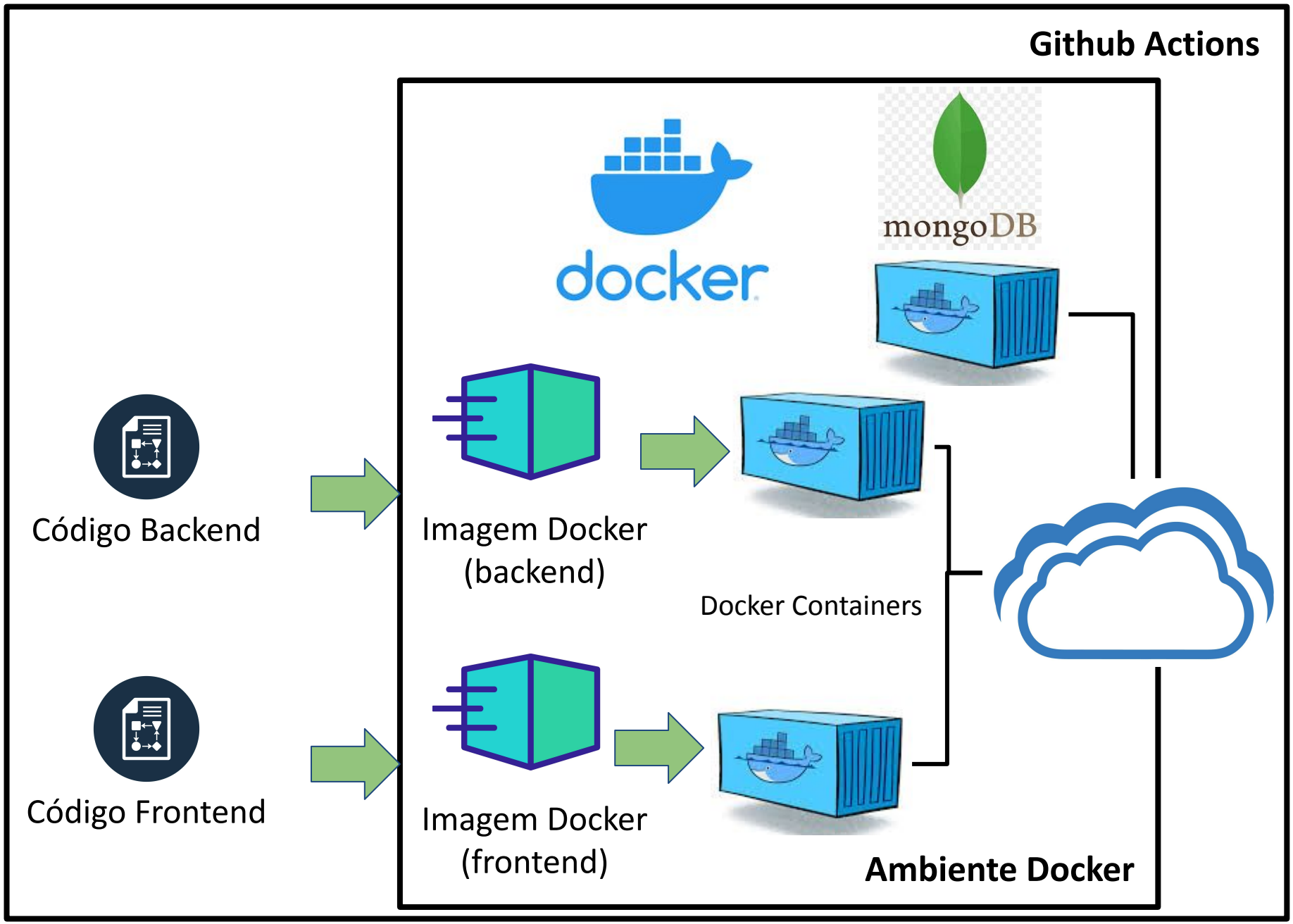
Exemplo prático

NodeJS + React + Mongodb

O que vamos fazer?




Github Actions
docker
mongoDB
Código Backend
Imagem Docker
(backend)
Docker Containers
Código Frontend
Imagem Docker
(frontend)
Ambiente Docker

NodeJS

Projeto NodeJS

Macro etapas

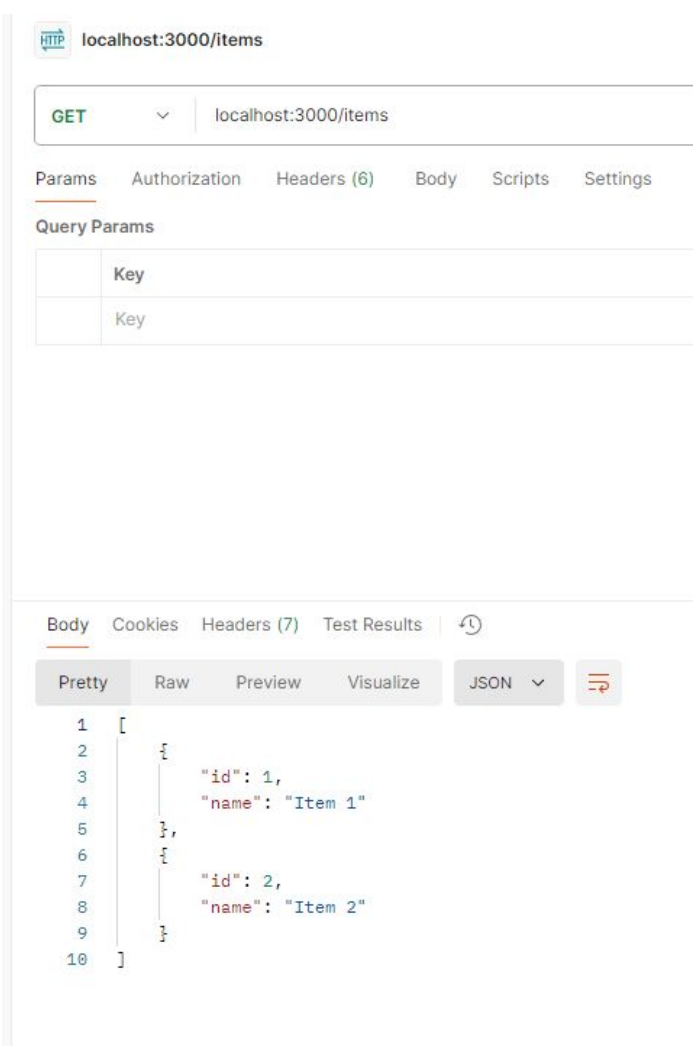
- Criar repositório no Github
  - https://github.com/silobocarvalho/gc-nodejs-app
- Criar o app.js com o código do servidor
  - Só isso já é o back-end :-)
- Instalar as dependências
  - npm install express cors
- Rodar o servidor
  - node app.js

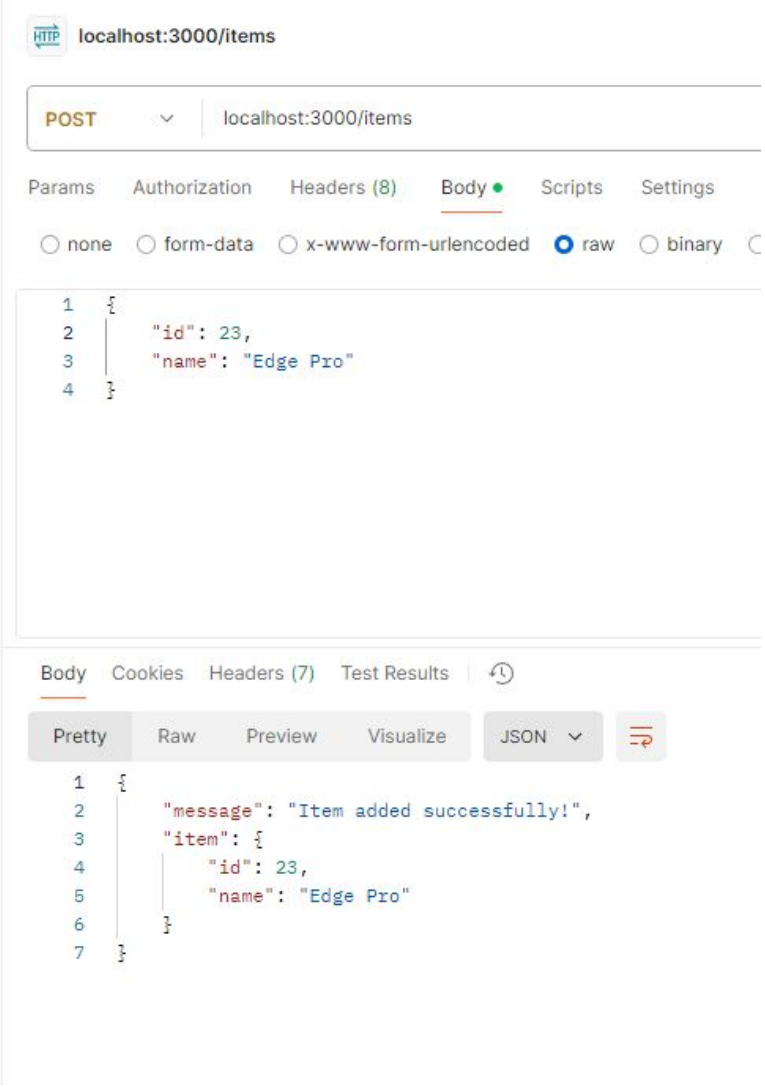
Projeto NodeJS

```
const express = require('express');
const app = express();
const PORT = 3000;
const cors = require('cors'); // Importa o pacote CORS

app.use(cors()); // Permitir requisicoes externas como as do React que iremos fazer
app.use(express.json()); // Middleware to parse JSON requests
// Routes
app.get('/', (req, res) => {
    res.send('Welcome to the Node.js API!');
});

app.get('/items', (req, res) => {
    const items = [
        { id: 1, name: 'Item 1' },
        { id: 2, name: 'Item 2' },
    ];
    res.json(items);
});

app.post('/items', (req, res) => {
    const newItem = req.body;
    res.status(201).json({ message: 'Item added successfully!', item: newItem });
});

app.listen(PORT, () => {
    console.log(`Server is running on http://localhost:${PORT}`);
});
```

```
C:\Users\Sidartha\Downloads\gc\backend>node app.js
Server is running on http://localhost:3000
```

Projeto NodeJS





Dockerfile NodeJS

- Crie um arquivo chamado Dockerfile (sem extensão mesmo)

```
# Use a imagem oficial do Node.js como base
FROM node:16

# Define o diretório de trabalho dentro do container
WORKDIR /app

# Instala o Express diretamente
RUN npm install express
RUN npm install cors

# Copia o arquivo app.js para o diretório de trabalho no container
COPY app.js .

# Exponha a porta utilizada pelo app
EXPOSE 3000

# Comando para iniciar o aplicativo
CMD ["node", "app.js"]
```

Dockerfile NodeJS

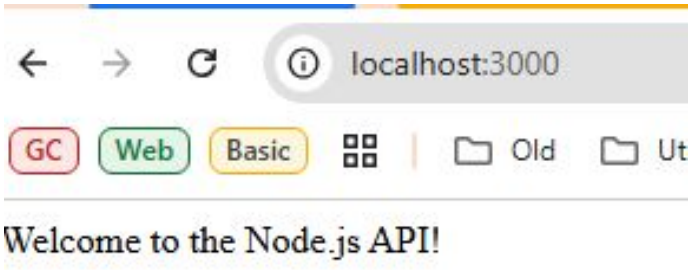
- Agora é só criar a imagem a partir do código
  - docker build -t nodejs-app .
    - Atenção ao ponto final (.) no final do comando

```
C:\Users\Sidartha\Downloads\gc\backend>docker build -t nodejs-app .
[+] Building 39.0s (10/10) FINISHED                                    docker:desktop-linux
 => [internal] load build definition from Dockerfile                                  0.1s
 => => transferring dockerfile: 429B                                                  0.0s
 => [internal] load metadata for docker.io/library/node:16                            2.3s
 => [auth] library/node:pull token for registry-1.docker.io                           0.0s
 => [internal] load .dockerignore                                                     0.0s
 => => transferring context: 2B                                                       0.0s
 => [1/4] FROM docker.io/library/node:16@sha256:f77a1aef2da8d83e45ec990f45df50f1a286c  33.3s
 => => resolve docker.io/library/node:16@sha256:f77a1aef2da8d83e45ec990f45df50f1a286c  0.0s
 => => sha256:ee7d78be1eb92caf6ae84fc3af736b23eca018d5dedc967ae5bdee6d7082403b 450B / 450B  0.4s
 => => sha256:ca266fd6192108b67fb57b74753a8c4ca5d8bd458baae3d4df7ce9f42dedcc1d 2.27MB / 2.27MB  0.8s
 => => sha256:0e421f66aff42bb069dffc26af6d132194b22a1082b08c5ef7cd69c627783c04 34.79MB / 34.79MB  6.5s
 => => sha256:ae3b95bbaa61ce24cefdd89e7c74d6fbd7713b2bcae93af47063d06bd7e02172 4.20kB / 4.20kB  0.6s
 => => sha256:513d779256048c961239af5f500589330546b072775217272e19ffae1635e98e 191.90MB / 191.90MB  29.4s
 => => sha256:ffd9397e94b74abcb54e514f1430e00f604328d1f895eadbd482f08cc02444e5 51.89MB / 51.89MB  15.6s
 => => sha256:7e9bf114588c05b2df612b083b96582f3b8dbf51647aa6138a50d09d42df2454 17.58MB / 17.58MB  4.5s
```

Dockerfile NodeJS

- Depois, executar o container localmente
  - docker run -p 3000:3000 nodejs-app

```
C:\Users\Sidartha\Downloads\gc\backend>docker run -p 3000:3000 nodejs-app
Server is running on http://localhost:3000
```



package.json

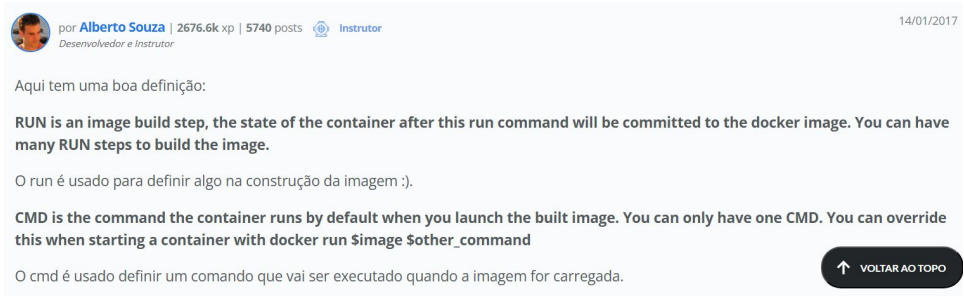
- Repare que como nosso projeto é simples, só fizemos uso do Express como dependência, mas se tivermos um projeto que faz uso de outras, é importante criar o ***package.json*** com as dependências do projeto.
- Para criar o ***package.json***
  - npm init
  - npm install express

```
{
  "name": "nodejs-app",
  "version": "1.0.0",
  "main": "app.js",
  "scripts": {
    "test": "echo \"Error: no test
specified\" && exit 1"
  },
  "author": "",
  "license": "ISC",
  "description": "",
  "dependencies": {
    "express": "^4.21.2"
  }
}
```

Dockerfile NodeJS

- Agora precisamos alterar nosso Dockerfile para incluir o package.json na criação da imagem
- Toda dependência agora é instalada automaticamente

```
# Use the official Node.js LTS image as the base image
FROM node:16

# Set the working directory inside the container
WORKDIR /app

# Copy package.json and package-lock.json to the working directory
COPY package*.json ./

# Install the dependencies
RUN npm install

# Copy the rest of the application code
COPY . .

# Expose the port the app runs on
EXPOSE 3000

# Start the application
CMD ["node", "app.js"]
```



Dockerfile NodeJS

- Criar a imagem novamente
  - docker build -t nodejs-app .

```
C:\Users\Sidartha\Downloads\gc\backend>docker build -t nodejs-app .
[+] Building 6.6s (11/11) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 465B
 => [internal] load metadata for docker.io/library/node:16
 => [auth] library/node:pull token for registry-1.docker.io
 => [internal] load .dockerignore
```

- docker run -p 3000:3000 nodejs-app

```
C:\Users\Sidartha\Downloads\gc\backend>docker run -p 3000:3000 nodejs-app
Server is running on http://localhost:3000
```


Backend criado com sucesso!
meow_code

React

Projeto React

- Criar um projeto React com o Vite
  - O vite é uma ferramenta que auxilia na criação da estrutura do projeto React
  - Há outros como
    - Next.js, Remix, Create React App (CRA), etc

- Para criar o projeto
  - npm create vite@latest

- Para executar
  - npm install
  - npm run dev

Dockerfile React

```
# Etapa 1: Construir o app React com Vite | Se usar uma ferramenta diferente
para criar o projeto React, as pastas usadas no COPY podem mudar!
FROM node:18 AS build

WORKDIR /app

# Copiar os arquivos de configuração
COPY package.json package-lock.json ./

# Instalar dependências
RUN npm install

# Copiar o restante do projeto
COPY . .

# Executar o build com Vite
RUN npm run build

# Verificar a estrutura do diretório após o build (opcional)
RUN ls -l /app

# Etapa 2: Servir os arquivos estáticos com Nginx
FROM nginx:stable-alpine

# Copiar os arquivos de build para o diretório padrão do Nginx
COPY --from=build /app/dist /usr/share/nginx/html

# Expor a porta
EXPOSE 80

# Iniciar o Nginx
CMD ["nginx", "-g", "daemon off;"]
```

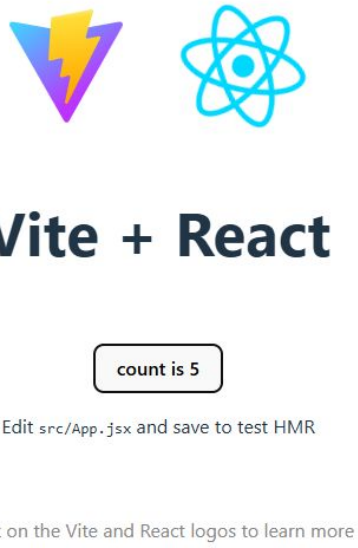
Projeto React

- Adicionar no package.json se usando “create-react-app”

```
"scripts": {
  "start": "react-scripts start",
  "build": "react-scripts build",
  "test": "react-scripts test",
  "eject": "react-scripts eject"
}
```

Projeto React

- **Criar a imagem**
  - docker build -t react-app .
- **Criar container**
  - docker run -p 8080:80 react-app
- Abrir o navegador http://localhost:8080



Projeto React

- **Modificando o projeto React para acessar a nossa API criada com NodeJS**

```
import { useState, useEffect } from 'react';

function App() {
  const [items, setItems] = useState([]);
  const [newItem, setNewItem] = useState('');
  const [message, setMessage] = useState('');

  // Função para obter os itens da API
  useEffect(() => {
    fetch('http://localhost:3000/items')
      .then((response) => response.json())
      .then((data) => {
        setItems(data);
      })
      .catch((error) => console.error('Error fetching items:', error));
  }, []);

  // Função para adicionar um novo item
  const handleAddItem = () => {
    const item = { name: newItem };

    fetch('http://localhost:3000/items', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
      },
      body: JSON.stringify(item),
    })
      .then((response) => response.json())
      .then((data) => {
        setMessage(data.message);
        setItems((prevItems) => [...prevItems, data.item]);
        setNewItem(''); // Limpa o campo de entrada
      })
      .catch((error) => console.error('Error adding item:', error));
  };

  return (
    <div>
      <h1>React App with Node.js API</h1>

      <h2>Items List</h2>
      <ul>
        {items.map((item) => (
          <li key={item.id}>{item.name}</li>
        ))}
      </ul>

      <div>
        <h2>Add New Item</h2>
        <input
          type="text"
          value={newItem}
          onChange={(e) => setNewItem(e.target.value)}
          placeholder="Enter item name"
        />
        <button onClick={handleAddItem}>Add Item</button>
      </div>

      {message && <p>{message}</p>}
    </div>
  );
}

export default App;
```

https://github.com/silobocarvalho/gc-react-app

Projeto React

- **Recriando a imagem a partir do novo código…**
  - docker build -t react-app .

Projeto NodeJS + React

- Subindo a imagem do backend NodeJS e do Frontend React
  - docker run -p 3000:3000 nodejs-app
  - docker run -p 8080:80 react-app


Frontend criado!

Mongodb + docker-compose

Mongodb

- Parece que já está complicado o suficiente, né? Mas ainda falta o banco de dados

https://github.com/silobocarvalho/gc-nodejs-app/tree/add-mongodb

Mongodb

- Criar o docker-compose.yml

```yaml
version: '3.8'
services:
  mongo:
    image: mongo:6.0
    container_name: mongo
    ports:
      - '27017:27017'
    volumes:
      - mongo-data:/data/db
    restart: always

  nodejs-app:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: nodejs-app
    ports:
      - '3000:3000'
    depends_on:
      - mongo
    environment:
      - MONGO_URI=mongodb://mongo:27017/mydatabase
    volumes:
      - .:/usr/src/app
    command: node app.js

volumes:
  mongo-data:
```

Mongodb

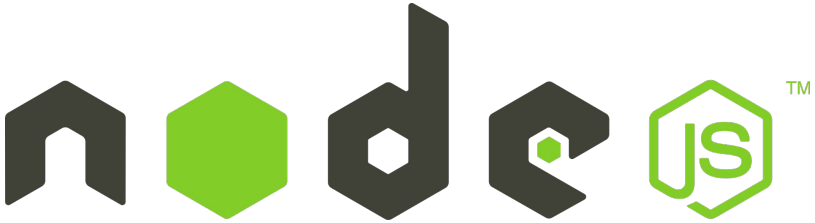
Para iniciar o backend Nodejs e o Mongodb é só subir o docker-compose

docker-compose up --build

O –build serve para rebuildar as imagens e evitar problemas com cache.

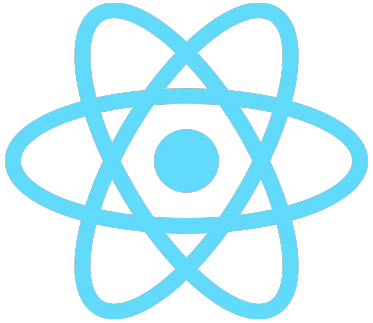

Banco de Dados criado!
Relational Databases

Docker Compose



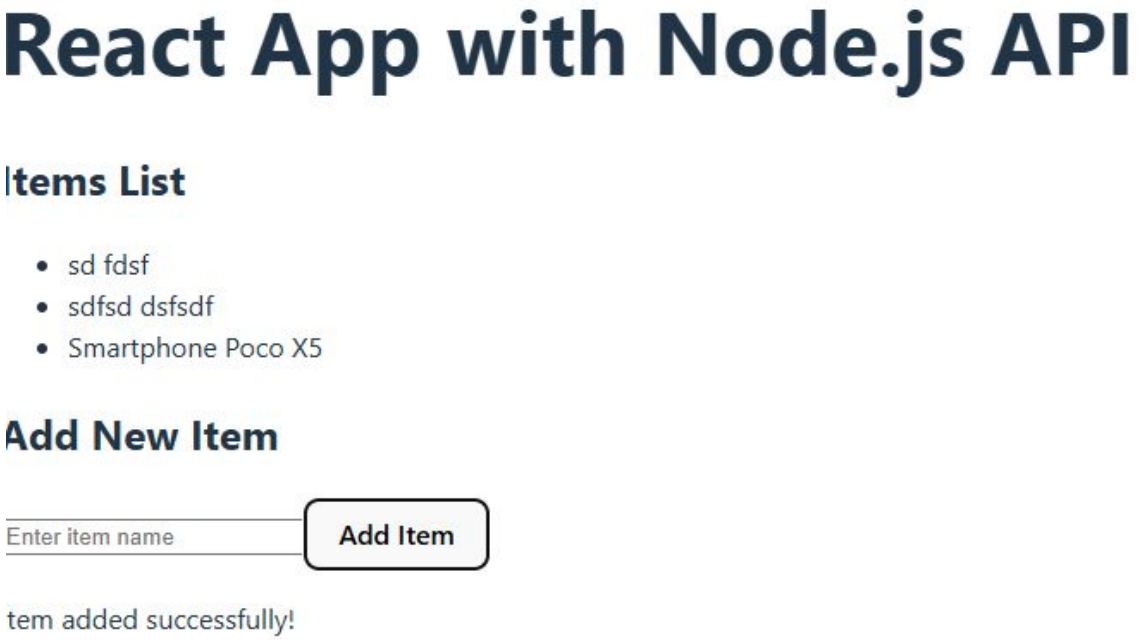
Aplicação completa com backend, frontend e banco de dados containerizada no Docker e executando na pipeline do Github Actions!

Ainda não!



Docker Compose

	Name	Image	
☐	backend	-	-
☐	nodejs-app	backend-node-app	3000:3000
☐	mongo	mongo:6.0	27017:27017
☐	wizardly_moser	react-app	8080:80



Criando testes

CI/CD: Criando testes

- Antes de automatizar, vamos adicionar alguns testes no nosso projeto… Os testes servem para evitar que novas funcionalidades interfiram ou quebrem outros trechos de código.

- Logo, sempre que for feito um novo push para o repositório, devemos assegurar que não irá quebrar o sistema.

CI/CD: Criando testes

- Criar um arquivo test.js

```
const http = require('http');
const assert = require('assert');

// Função para fazer requisições HTTP
function makeRequest(options, data = null) {
  return new Promise((resolve, reject) => {
    const req = http.request(options, (res) => {
      let body = '';
      res.on('data', (chunk) => (body += chunk));
      res.on('end', () => resolve({ statusCode: res.statusCode, body }));
    });
    req.on('error', reject);
    if (data) req.write(data);
    req.end();
  });
}

// Testes
(async () => {
  console.log('Running tests...');

  try {
    // Teste 1: Rota raiz (GET /)
    const responseRoot = await makeRequest({
      hostname: 'localhost',
      port: 3000,
      path: '/',
      method: 'GET',
    });

    assert.strictEqual(responseRoot.statusCode, 200, 'GET / should return 200');
    assert.strictEqual(responseRoot.body, 'Welcome to the Node.js API!', 'GET / should return welcome message');

    console.log('✔ Test 1: GET / passed');

    // Teste 2: Adicionar item (POST /items)
    const newItem = JSON.stringify({ name: 'Test Item' });
    const responsePost = await makeRequest(
      {
        hostname: 'localhost',
        port: 3000,
        path: '/items',
        method: 'POST',
        headers: {
          'Content-Type': 'application/json',
          'Content-Length': Buffer.byteLength(newItem),
        },
      },
      newItem
    );

    assert.strictEqual(responsePost.statusCode, 201, 'POST /items should return 201');
    const responseBodyPost = JSON.parse(responsePost.body);
    assert.strictEqual(responseBodyPost.message, 'Item added successfully!', 'POST /items should return success message');
    assert.strictEqual(responseBodyPost.item.name, 'Test Item', 'POST /items should return the created item');

    console.log('✔ Test 2: POST /items passed');

    // Teste 3: Listar itens (GET /items)
    const responseGetItems = await makeRequest({
      hostname: 'localhost',
      port: 3000,
      path: '/items',
      method: 'GET',
    });

    assert.strictEqual(responseGetItems.statusCode, 200, 'GET /items should return 200');
    const items = JSON.parse(responseGetItems.body);
    assert.ok(Array.isArray(items), 'GET /items should return an array');
    assert.ok(items.length > 0, 'GET /items should return at least one item');

    console.log('✔ Test 3: GET /items passed');
  } catch (error) {
    console.error('Test failed:', error.message);
    process.exit(1);
  }

  console.log('All tests passed successfully!');
  process.exit(0);
})();
```

CI/CD: Criando testes

- Alterar o package.json

```
"scripts": {
  "start": "react-scripts start",
  "build": "react-scripts build",
  "test": "node test",
  "eject": "react-scripts eject"
},
```

- Subir nossa aplicação com o docker compose
  - docker-compose up

```
> nodejs-app@1.0.0 test
> node test

Running tests...
✔Test 1: GET / passed
✔Test 2: POST /items passed
Test failed: GET /items should return 200
PS C:\Users\Sidartha\Downloads\gc\backend> D
```

- Executar os testes
  - npm test

```
PS C:\Users\Sidartha\Downloads\gc\backend> npm test

> nodejs-app@1.0.0 test
> node test

Running tests...
✔Test 1: GET / passed
✔Test 2: POST /items passed
✔Test 3: GET /items passed
All tests passed successfully!
```

Automatizando com Github Actions

CI/CD: Github Actions

- Ao adicionar um arquivo em uma localização especifica, o Github identifica este arquivo e já cria as actions para o nosso projeto.

- Crie o arquivo .github/workflows/test.yml

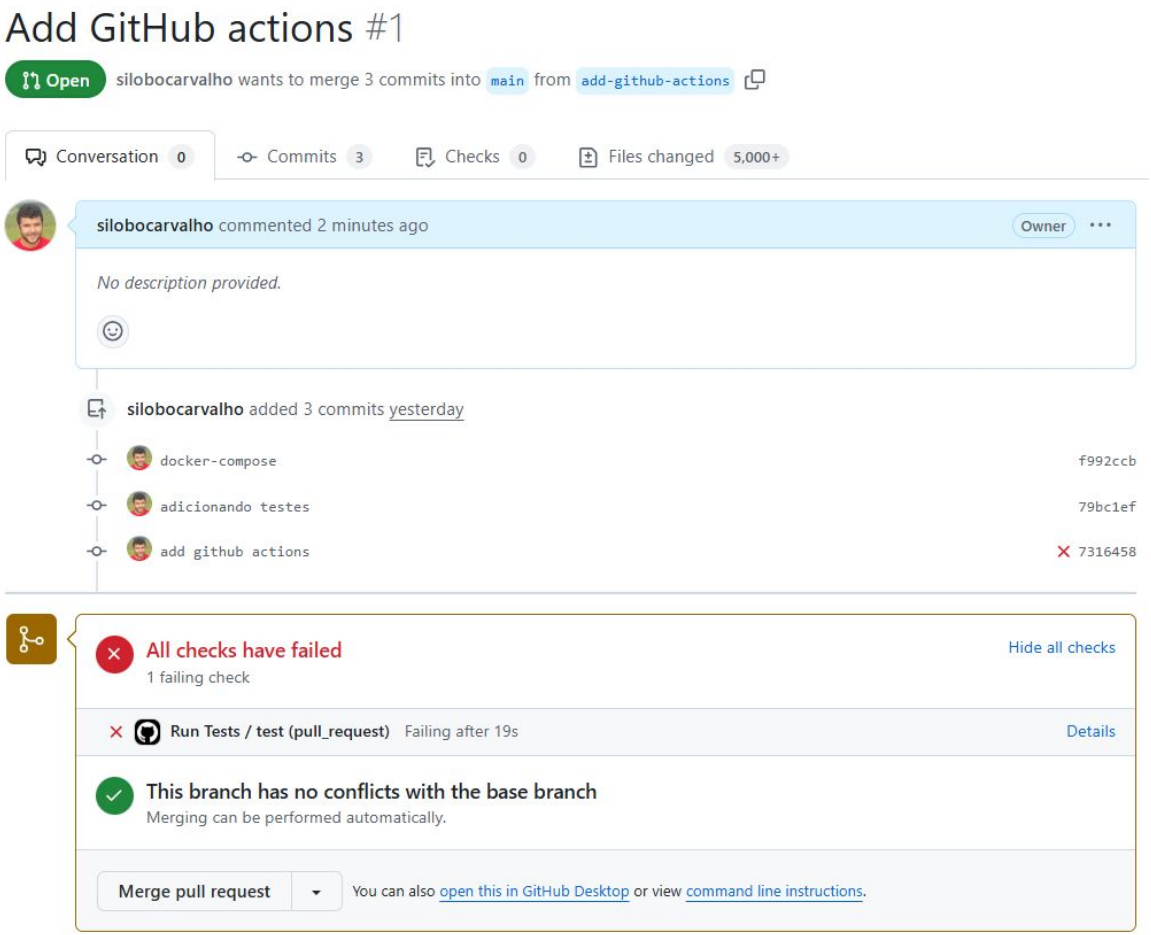
CI/CD: Github Actions

- Crie o arquivo .github/workflows/test.yml

```yaml
name: Run Tests

on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main

jobs:
  test:
    runs-on: ubuntu-latest

    services:
      mongo:
        image: mongo:latest
        ports:
          - 27017:27017

    steps:
      - name: Checkout code
        uses: actions/checkout@v3

      - name: Set up Node.js
        uses: actions/setup-node@v3
        with:
          node-version: 18

      - name: Install dependencies
        run: npm install

      - name: Run tests
        run: npm test
```

CI/CD: Github Actions

- Ao abrir um Pull Request para a Main, dentro do PR a pipeline já vai executar…



CI/CD: Github Actions

- Se você acertar de primeira, estranhe…

- Após alguns testes e validações, chegamos ao arquivo final do Github Actions

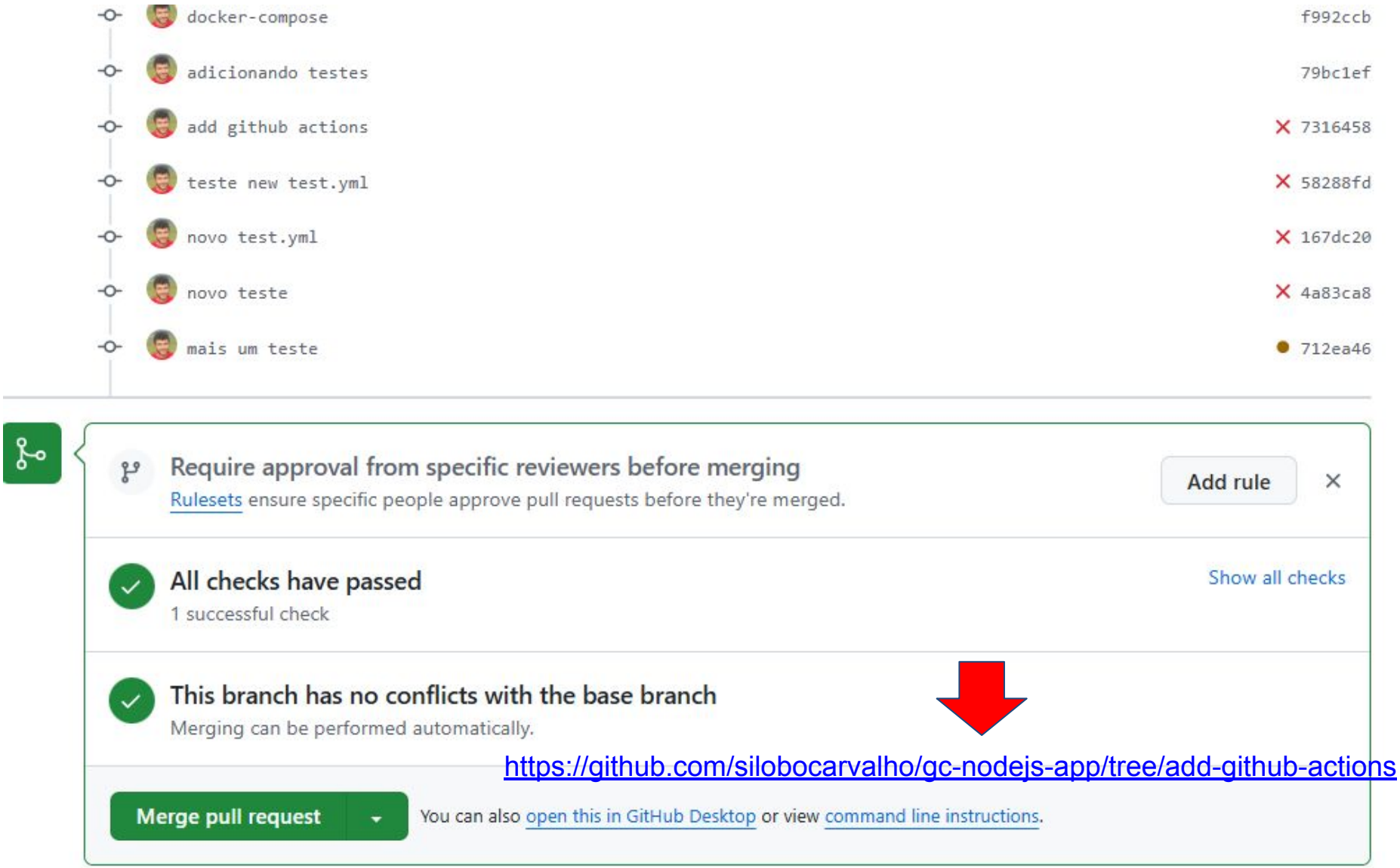
CI/CD: Github Actions

```yaml
name: Node.js CI

on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
    - name: Check out code
      uses: actions/checkout@v3

    - name: Set up Node.js
      uses: actions/setup-node@v3
      with:
        node-version: '18'

    - name: Install Docker Compose
      run: |
        sudo apt-get update
        sudo apt-get install -y docker-compose

    - name: Install dependencies
      run: npm install

    - name: Start Docker Compose
      run: docker-compose up -d

    - name: Wait for services to be ready
      run: |
        for i in {1..60}; do
          if nc -z localhost 3000; then
            echo "Server is ready!"
            exit 0
          fi
          echo "Waiting for server..."
          sleep 2
        done
        echo "Server failed to start in time"
        exit 1

    - name: Check Docker logs
      run: docker-compose logs

    - name: Run tests
      run: npm test

    - name: Stop Docker Compose
      if: always()
      run: docker-compose down
```

```yaml
name: Node.js CI

on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - name: Check out code
        uses: actions/checkout@v4

      - name: Set up Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '18'

      - name: Install Docker Compose (if not present)
        run: |
          if ! command -v docker-compose >/dev/null 2>&1; then
            sudo apt-get update
            sudo apt-get install -y docker-compose
          else
            echo "docker-compose already installed"
          fi

      - name: Install dependencies
        run: npm ci

      - name: Start Docker Compose
        run: docker-compose up -d

      - name: Wait for server to be ready (HTTP)
        uses: jakejarvis/wait-action@v0.2.0
        with:
          url: http://localhost:3000
          timeout: 120   # tempo máximo em segundos

      - name: Run tests
        run: npm test

      - name: Check Docker logs (only on failure)
        if: failure()
        run: docker-compose logs --no-color

      - name: Stop Docker Compose
        if: always()
        run: docker-compose down
```

TESTAR ESSA NOVA SOLUCAO SEM A GAMBIARRA DO FOR

CI/CD: Github Actions



https://github.com/silobocarvalho/gc-nodejs-app/tree/add-github-actions

CI/CD: Github Actions

- **Fizemos muitas coisas até aqui…**

- Criamos um servidor NodeJs para o back-end
- Criamos um aplicativo React para o front-end, para consumir os dados do back-end
- Criamos um container para persistir os dados no banco de dados Mongodb
- Criamos testes para nossa aplicação back-end
- Automatizamos a execução dos testes
- Automatizamos a build, criação da imagem Docker, execução dos sistemas (docker-compose) e a execução dos testes (npm test) com o Github Actions…
  - ufaaaaa


I'M FINE
OUT OF ORDER

Dúvidas?